MiniDecaf Compiler Lab Report - Stage 1

Student ID: 2021080066

Name: 罗兰 Ruslan Bayramov

Overview

In Stage 1 of the MiniDecaf Compiler Lab, I implemented support for various unary and binary operators, as well as parentheses, in the MiniDecaf language. The work was divided into several steps:

- **Step 2**: Implemented unary operators: negation (`-`), bitwise NOT (`~`), and logical NOT (`!`).
- **Step 3**: Implemented binary arithmetic operators: addition (`+`), subtraction (`-`), multiplication (`*`), division (`/`), modulus (`%`), and parentheses `(` and `)`.
- **Step 4**: Implemented relational operators (`<` , `>` , `<=` , `>=`), equality operators (`==` , `!=`), logical AND (`&&`), and logical OR (`||`).

The implementation involved modifying the lexer and parser to recognize the new operators and update the grammar to reflect operator precedence and associativity. Additionally, I updated the semantic analysis and intermediate code generation to handle the new operations, and modified the backend to generate the appropriate RISC-V assembly code.

Implementation Details

Lexer and Parser Updates

I extended the MiniDecaf grammar to include the new unary and binary operators, following the C language precedence and associativity rules. The updated grammar ensures correct parsing of expressions involving multiple operators and parentheses.

For example, the grammar for expressions was updated to:

```
expression
    : logical_or

logical_or
    : logical_and
    | logical_or '||' logical_and

logical_and
    : equality
    | logical_and '&&' equality
```

```
equality
    : relational
    | equality ('=='|'!=') relational

relational
    : additive
    | relational ('<'|'>'|'<='|'>=') additive

additive
    : multiplicative
    | additive ('+'|'-') multiplicative

multiplicative
    : unary
    | multiplicative ('*'|'/'|'%') unary

unary
    : primary
    | ('-'|'~'|'!') unary

primary
    : Integer
    | '(' expression ')'
```

Semantic Analysis

I updated the semantic analysis phase to correctly handle the new operators, ensuring that type checking and expression evaluation semantics are correctly applied.

Intermediate Code Generation

I extended the TAC (Three-Address Code) representation to include new instructions for the added operators, such as:

- **Unary Operations**:
  - `NEG dst, src` : Negation
  - `BITNOT dst, src` : Bitwise NOT
  - `LOGICNOT dst, src` : Logical NOT
- **Binary Operations**:
  - `ADD dst, src1, src2`
  - `SUB dst, src1, src2`
  - `MUL dst, src1, src2`
  - `DIV dst, src1, src2`
  - `MOD dst, src1, src2`
  - `BEQ dst, src1, src2` : Equality comparison (`==`)
  - `BNE dst, src1, src2` : Inequality comparison (`!=`)
  - `SLT dst, src1, src2` : Less than (`<`)
  - `SGT dst, src1, src2` : Greater than (`>`)
  - `BLE dst, src1, src2` : Less than or equal to (`<=`)
  - `BGE dst, src1, src2` : Greater than or equal to (`>=`)
  - `LAND dst, src1, src2` : Logical AND
  - `LOR dst, src1, src2` : Logical OR

I implemented the logic for generating these TAC instructions from the AST nodes corresponding to the new operators.

Backend Code Generation

In the backend, I updated the RISC-V assembly emitter to translate the new TAC instructions into appropriate RISC-V assembly instructions. This involved adding new cases to handle the new unary and binary operations, ensuring correct assembly code is generated.

For example, the logical AND (`&&`) and logical OR (`||`) operations were implemented without short-circuit evaluation, as per the lab requirements. Here is how the `LAND` and `LOR` operations were translated:

- **Logical OR (** `LOR` **)**:

```
or dst, src1, src2
snez dst, dst
```

- **Logical AND (** `LAND` **)**:

```
snez dst, src1
sub dst, zero, dst
and dst, dst, src2
snez dst, dst
```

For relational operators, such as less than or equal (`BLE`), which is not directly provided in RISC-V, I used combinations of instructions to achieve the desired result.

Thinking Questions

Question: In MiniDecaf, we don't require short-circuit evaluation for logical operators, but in most popular languages including C, short-circuit evaluation is supported. Why is this feature popular? What benefits does short-circuit evaluation bring to programmers?

Answer:

Short-circuit evaluation is popular because it allows logical expressions to be evaluated efficiently and safely by not evaluating unnecessary expressions. In logical AND (`&&`) and logical OR (`||`) operations, the result can often be determined by evaluating only the first operand:

- For `&&` , if the first operand is `false` (or `0` in C), the entire expression is `false` , and the second operand does not need to be evaluated.
- For `||` , if the first operand is `true` (or non-zero in C), the entire expression is `true` , and the second operand does not need to be evaluated.

Benefits of short-circuit evaluation:

1. **Performance Improvement**: By avoiding the evaluation of the second operand when not necessary, programs can run faster, especially if the second operand involves complex computations or function calls.
2. **Avoiding Side Effects**: In expressions where the second operand has side effects (e.g., function calls that modify variables or perform I/O), short-circuit evaluation prevents these side effects from occurring when they are not needed. This allows programmers to write expressions that are both concise and safe.
3. **Preventing Errors**: Short-circuit evaluation can prevent runtime errors by avoiding the evaluation of expressions that may cause exceptions or undefined behavior. For example, in the expression `(ptr != NULL) && (ptr->field == value)`, the second operand dereferences `ptr`. If `ptr` is `NULL`, short-circuit evaluation prevents the dereferencing of `NULL`, avoiding a segmentation fault.
4. **Logical Control Flow**: It enables the use of logical operators to control the flow of execution in expressions, making code more readable and expressive.

Overall, short-circuit evaluation provides both efficiency and safety advantages, making it a valuable feature in programming languages.